



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA -2
Academic year	I – III Semester(2025-26)
Faculty Name	Mohammed Razia Alangir Banu
Student Name	B.Deepak Goud
Roll Number	2520080035
Branch	AI&DS
Course Outcome	3

Case Study Topic : InstaShare — story visibility prediction system

InstaShare predicts the short-term visibility of a newly uploaded story.

When a creator uploads a story, the platform performs a **bounded Breadth First Search (BFS)** on the follower graph up to depth 3 because users beyond 3 hops rarely interact with the story.

The graph contains overlapping follower communities, meaning some users can be reached through multiple paths. Therefore, the platform must use a **visited-set** to avoid duplicate counting.

The compute budget for prediction is:

p99 latency < 400 ms

Miniature Example Graph

Author/source user:

P

Edges:

P→Q

P→R

Q→S

Q→T

R→T

R→U

S→V

T→V

T→W

U → W

Depth Layout

Depth 0 : P

Depth 1 : Q, R

Depth 2 : S, T, U

Depth 3 : V, W

Important Overlaps

- T reachable from both Q and R

- V reachable from both S and T
- W reachable from both T and U

Without a visited-set, BFS would double-count users.

Questions:

(a)

Run BFS from P bounded to depth 3. Show the queue evolution after every iteration: current node removed, depth, neighbours discovered, and visited-set contents. Process neighbours alphabetically. Then state: (i) the final set of reached users excluding P, and (ii) for every multiply-reachable node, identify which predecessor first discovered it and explain how the visited-set prevents duplicate counting.

(b)

Complete the Java boilerplate below so that `bfsLimited(adj, source, maxDepth)` returns all users reachable within the bounded depth using a queue and visited-set. The solution must run in $O(V + E)$ over the explored subgraph. Fill in the missing TODO sections for pruning, neighbour expansion, and result accumulation.

```
import java.util.*;

static Set<String> bfsLimited(
    Map<String, List<String>> adj,
    String source,
    int maxDepth) {

    Set<String> visited = new HashSet<>();
    visited.add(source);

    Deque<Object[]> queue =
        new ArrayDeque<>();

    queue.offer(new Object[]{ source, 0 });

    Set<String> reached =
        new HashSet<>();

    while (!queue.isEmpty()) {
        Object[] cur = queue.poll();

        String u = (String) cur[0];
        int depth = (int) cur[1];

        // TODO 1:
        // if depth == maxDepth,
        // do not expand neighbours

        // TODO 2:
        // explore neighbours

        // TODO 3:
        // add newly visited users
        // into reached
    }

    return reached;
}
```

(c)

Capacity analysis under the 400 ms p99 budget. Suppose the creator has 40,000 direct followers (depth-1), and each follower has on average 300 followers. Assume about 25% overlap among candidate users because of shared audiences. Estimate: (i) distinct users reached within $\text{depth} \leq 3$ using a visited-set, (ii) total work without a visited-set, and (iii) whether the bounded BFS remains feasible under the assumption that each neighbour visit costs approximately 1.5 μs .

Final Reached Users

Excluding source P:

{Q, R, S, T, U, V, W}

Total reached users:

7

Multiple-Predecessor Users

User T

- First reached via Q
 - When R attempts to reach T, visited-set prevents duplicate enqueue
-

User V

- First reached via S
 - T's attempt to revisit V is ignored
-

User W

- First reached via T
 - U's duplicate attempt is rejected
-

Why Visited-Set Matters

Without the visited-set:

- T would be added twice
- V would be added twice
- W would be added twice

In large social graphs, such duplicate expansions create exponential queue growth and massive unnecessary computation.

(b) Java BFS Code

```

1  import java.util.*;
2
3  public class BFSLimited {
4      static Set<String> bfslimited(
5          Map<String, List<String>> adj,
6          String source,
7          int maxDepth) {
8
9          Set<String> visited = new HashSet<>();
10
11          visited.add(source);
12
13          Deque<Object[]> queue =
14              new ArrayDeque<>();
15
16          queue.offer(new Object[]{ source, 0 });
17
18          Set<String> reached =
19              new HashSet<>();
20
21          while (!queue.isEmpty()) {
22              Object[] cur = queue.poll();
23
24              String u = (String) cur[0];
25              int depth = (int) cur[1];
26
27              // Add non-source users
28              if (!u.equals(source))
29                  reached.add(u);
30
31              // Stop expansion at depth limit
32              if (depth == maxDepth)
33                  continue;
34
35              // Explore neighbours
36              for (String v :
37                  adj.getOrDefault(
38                      u,
39                      Collections.emptyList())) {
40                  if (!visited.contains(v)) {
41                      visited.add(v);
42                      queue.offer(
43                          new Object[]{ v, depth + 1 });
44                  }
45              }

```

```

3  public class BFSLimited {
5      static Set<String> bfslimited(
47
48          queue.offer(
49              new Object[]{
50                  v,
51                  depth + 1
52              });
53      }
54  }
55
56      return reached;
57  }
58
59
60  public static void main(String[] args) {
61
62      Map<String, List<String>> graph =
63          new HashMap<>();
64
65      graph.put(key: "P", Arrays.asList(...: "Q", "R"));
66      graph.put(key: "Q", Arrays.asList(...: "S", "T"));
67      graph.put(key: "R", Arrays.asList(...: "T", "U"));
68      graph.put(key: "S", Arrays.asList(...: "V"));
69      graph.put(key: "T", Arrays.asList(...: "V", "W"));
70      graph.put(key: "U", Arrays.asList(...: "W"));
71
72      Set<String> result =
73          bfslimited(graph, source: "P", maxDepth: 3);
74
75      System.out.println(result);
76  }
79  }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS D:\CaseStudy Implementation> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\kireeti\AppData\Roaming\workspaceStorage\936ae44d07e7b9219e05e7f687c48005\redhat.java\jdt_ws\CaseStudy Implementation_6f20df2\bin' 'BFSLimited'
[Q, R, S, T, U, V, W]
PS D:\CaseStudy Implementation> ^C
PS D:\CaseStudy Implementation>
PS D:\CaseStudy Implementation> d; cd 'd:\CaseStudy Implementation'; & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages'
'C:\Users\kireeti\AppData\Code\User\workspaceStorage\936ae44d07e7b9219e05e7f687c48005\redhat.java\jdt_ws\CaseStudy Implementation_6f20df2\bin' 'BFSLimited'
[Q, R, S, T, U, V, W]
PS D:\CaseStudy Implementation>

```

Complexity Analysis

Each node is visited at most once because of the visited-set.

Time complexity:

$$O(V + E)$$

where:

- V = visited users
- E = explored edges

Depth pruning ensures BFS never expands beyond:

$$\text{maxDepth}$$

(c) Capacity Analysis

Given

- Depth-1 followers:
40,000
- Average followers per user:
300
- Overlap reduction:
25%
- Per neighbour visit cost:
 $1.5\mu s$

(i) Distinct Users with Visited-Set

Depth 1

$$40,000$$

Depth 2

Raw candidates:

$$40,000 \times 300 = 12,000,000$$

After removing 25% overlap:

$$12,000,000 \times 0.75 = 9,000,000$$

Depth 3

Raw candidates:

$$9,000,000 \times 300 = 2,700,000,000$$

After overlap reduction:

$$2,700,000,000 \times 0.75 = 2,025,000,000$$

Total Distinct Reach

$$40,000 + 9,000,000 + 2,025,000,000 \approx 2.034 \text{ billion users (ii) Work Without Visited – Set}$$

Naive BFS explores every edge repeatedly.

Approximate operations:

$$40,000 \times 300 \times 300 = 3.6 \text{ billion expansions}$$

Queue growth becomes extremely large because repeated users are continuously re-enqueued.

(iii) Feasibility Under 400 ms Budget

Using visited-set:

$$2.034 \times 10^9 \times 1.5 \mu s = 3.051 \times 10^9 \mu s = 3051 \text{ seconds} \approx 50.8 \text{ minutes}$$

Conclusion

Exact bounded BFS is completely infeasible at production social-network scale.

Even with overlap pruning and visited-set optimization, the workload exceeds the:

400 ms p99 SLA

by several orders of magnitude.

Production systems instead use:

- Sampling
- Probabilistic estimators
- Reach prediction models
- Graph sketches such as HyperLogLog
- Statistical extrapolation

instead of exact BFS traversal.

GitHub Link:<https://github.com/bdeepakg-boop/DSA-Project/upload/main>

Student Signature	Faculty Signature